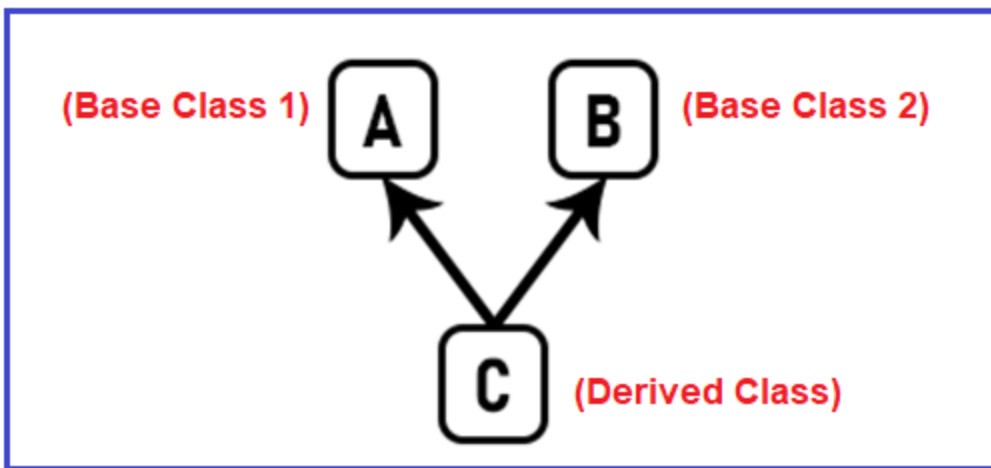


Multiple Inheritance Realtime Example in C#

In this article, I am going to discuss How to Use **Multiple Inheritances in C# Application Development with Realtime** Examples. Please read our previous article where we discussed [Multiple Inheritances in C#](#). If you are attending any interviews, then the interviewer might ask you this question what is the use of Multiple Inheritance in real-time applications or he might be asked you where you have used Multiple Inheritance in your project? Before understanding the use of Multiple Inheritance with the real-time example, let us first understand the concept of multiple inheritances.

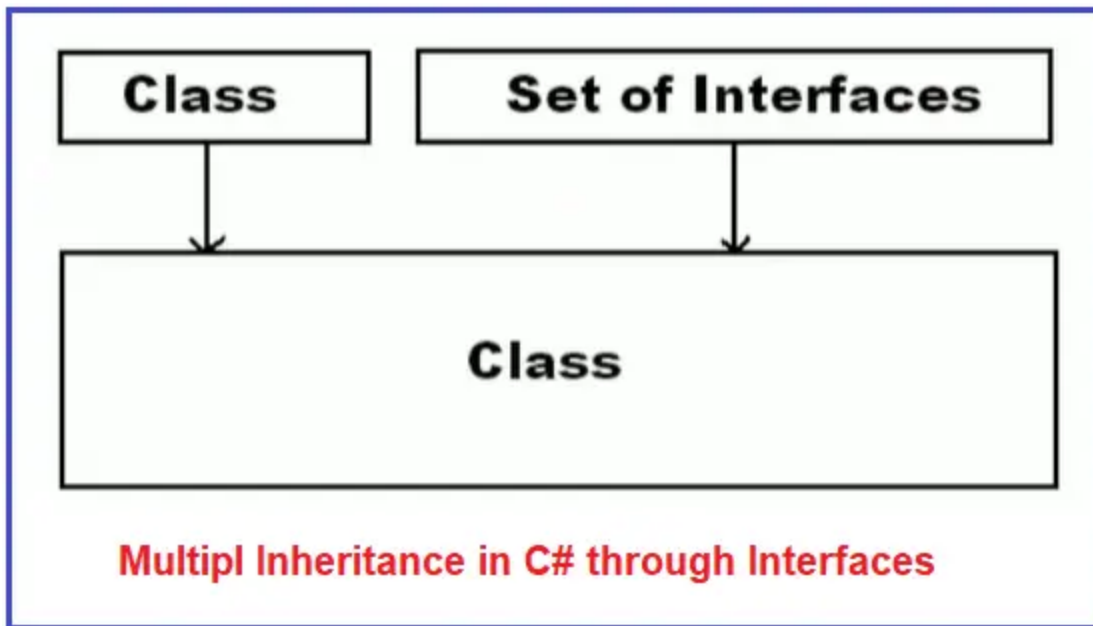
What is Multiple Inheritance in C#?

When a class is derived from more than one base class then such type of inheritance is called multiple inheritances. For a better understanding, please have a look at the below image.



As you can see in the above image, class C inherits from classes A and B, and for class C, we have two base or parent classes and this concept is not supported in C# with classes because of the ambiguity problem and that we have already discussed in our previous article.

But, in real-time we want to implement the above functionality and this is possible in C# with interfaces. Even if Multiple Inheritance is not supported through classes in C#, it is still supported through interfaces. A class can have one and only one immediate parent class, whereas the same class can have any number of interfaces as its parent i.e. Multiple Inheritance is supported in C# through interfaces. For a better understanding, please have a look at the below diagram.



As you can see in the above image, a class can have one and only one immediate parent class. At the same time, the same class can have n number of interfaces as its parent. So, the point that you need to remember is in C#, Multiple Inheritance is supported through interfaces in C#, not supported through classes. So, in this article, I am going to show you one real-time example where you need to go for multiple inheritances.

Example to Understand Multiple Inheritance in C#:

First, I will show you the example, without using multiple inheritances and then we will discuss the problem, and then we will see how we can overcome such problems using multiple inheritances in C# language.

Example without using Multiple Inheritance in C#

We need to develop one application to implement the printer service. So, as part of the printer service, we are going to provide the following four functionalities.

1. **Print**
2. **Fax**
3. **Scan**
4. **PrintDuplex**

So, you might be interested to declare one interface called `IPrinterTasks` with the above four as abstract methods as follows:

```
namespace MultipleInheritanceRealtimeExample
{
    public interface IPrinterTasks
    {
        void Print(string PrintContent);
        void Scan(string ScanContent);
        void Fax(string FaxContent);
        void PrintDuplex(string PrintDuplexContent);
    }
}
```

```

    }
}

```

As you can see in the above code, here, we have created an interface i.e. `IPrinterTasks` with four abstract methods. Now if any class wants to implement this interface, then that class should have to provide the implementation to all four methods of the `IPrinterTasks` interface.

We have two printers i.e. **HPLaserJetPrinter** and **LiquidInkjetPrinter** who wants the printer services. But the requirement is that the `HPLaserJetPrinter` wants all the services provided by the `IPrinterTasks` while the `LiquidInkjetPrinter` wants only two services i.e. Print and Scan service of the printer.

As we have declared all the methods within the `IPrinterTasks` interface, then it is mandatory for the `LiquidInkjetPrinter` class to provide implementation to Scan and Print methods along with the Fax and PrintDuplex method which are not required by the `LiquidInkjetPrinter` class.

HPLaserJetPrinter.cs

Create a class file with the name `HPLaserJetPrinter.cs` and then copy and paste the following code. Here, you can see that our class is inherited from the `IPrinterTasks` interface and provides implementation to all four interface methods.

```

using System;
namespace MultipleInheritanceRealtimeExample
{
    public class HPLaserJetPrinter : IPrinterTasks
    {
        public void Print(string PrintContent)
        {
            Console.WriteLine(PrintContent);
        }
        public void Scan(string ScanContent)
        {
            Console.WriteLine(ScanContent);
        }
        public void Fax(string FaxContent)
        {
            Console.WriteLine(FaxContent);
        }
        public void PrintDuplex(string PrintDuplexContent)
        {
            Console.WriteLine(PrintDuplexContent);
        }
    }
}

```

The above class required all the printer services and we provide implementation to all the four interface methods. That is fine.

LiquidInkjetPrinter.cs

Now, create a class file with the name LiquidInkjetPrinter.cs and then copy and paste the following code into it. Here, you can see that our class is also inherited from the IPrinterTasks interface and provides implementation to all four interface methods.

```
using System;
namespace MultipleInheritanceRealtimeExample
{
    class LiquidInkjetPrinter : IPrinterTasks
    {
        public void Print(string PrintContent)
        {
            Console.WriteLine(PrintContent);
        }
        public void Scan(string ScanContent)
        {
            Console.WriteLine(ScanContent);
        }
        public void Fax(string FaxContent)
        {
            throw new NotImplementedException();
        }
        public void PrintDuplex(string PrintDuplexContent)
        {
            throw new NotImplementedException();
        }
    }
}
```

The above class required only two printer services but here we are also providing implementation to all four interface methods. This is the problem. We should not provide implementation to the methods in which we are not interested. In this case, the class should not provide implementation to the Fax and PrintDuplex method.

How we can overcome this problem?

By splitting the above big interface with many small interfaces. For a better understanding, please have a look at the following code. As you can see in the below code, now we have split that one big interface into three small interfaces.

```
namespace MultipleInheritanceRealtimeExample
{
    public interface IPrinterTasks
    {
        void Print(string PrintContent);
        void Scan(string ScanContent);
    }
    interface IFaxTasks
    {
        void Fax(string content);
    }
}
```

```

interface IPrintDuplexTasks
{
    void PrintDuplex(string content);
}
}

```

Each interface now has some specific purpose. Now if any class wants all the services, then that class needs to implement all three interfaces. In our example, HPLaserJetPrinter wants all the services and hence, this class needs to be implemented in all three interfaces as shown in the below code. This is nothing but multiple inheritances. One class implementing multiple interfaces.

```

using System;
namespace MultipleInheritanceRealtimeExample
{
    public class HPLaserJetPrinter : IPrinterTasks, IFaxTasks, IPrintDuplexTasks
    {
        public void Print(string PrintContent)
        {
            Console.WriteLine(PrintContent);
        }
        public void Scan(string ScanContent)
        {
            Console.WriteLine(ScanContent);
        }
        public void Fax(string FaxContent)
        {
            Console.WriteLine(FaxContent);
        }
        public void PrintDuplex(string PrintDuplexContent)
        {
            Console.WriteLine(PrintDuplexContent);
        }
    }
}

```

Now, if any class wants the Scan and Print service, then that class needs to implement only the IPrinterTasks interfaces. In our example, LiquidInkjetPrinter wants only the Scan and Print services and hence this class needs to implement only the IPrinterTasks interface as shown in the below code.

```

using System;
namespace MultipleInheritanceRealtimeExample
{
    class LiquidInkjetPrinter : IPrinterTasks
    {
        public void Print(string PrintContent)
        {
            Console.WriteLine(PrintContent);
        }
        public void Scan(string ScanContent)
        {
            Console.WriteLine(ScanContent);
        }
    }
}

```

```

    }
}
}

```

Using the Printer Classes:

Now, once you create an instance of the HPLaserJetPrinter, then you can consume all the services. On the hand, if you create an instance of the LiquidInkjetPrinter, then you can consume only the Print and Scan services. Modify the Main method of the Program class as shown in the below image.

```

using System;
namespace MultipleInheritanceRealtimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            HPLaserJetPrinter hPLaserJetPrinter = new HPLaserJetPrinter();
            hPLaserJetPrinter.Scan("Scan Services by HPLaserJetPrinter");
            hPLaserJetPrinter.Print("Print Services by HPLaserJetPrinter");
            hPLaserJetPrinter.Fax("Fax Services by HPLaserJetPrinter");
            hPLaserJetPrinter.PrintDuplex("Print Duplex Services by HPLaserJetPrinter");
            LiquidInkjetPrinter liquidInkjetPrinter = new LiquidInkjetPrinter();
            liquidInkjetPrinter.Scan("Scan Services by LiquidInkjetPrinter");
            liquidInkjetPrinter.Print("Print Services by LiquidInkjetPrinter");
            //Fax and PrintDuplex are not available in LiquidInkjetPrinter
            //liquidInkjetPrinter.Fax("Fax Services");
            //liquidInkjetPrinter.PrintDuplex("Print Duplex Services");
            Console.Read();
        }
    }
}

```

Output:

```

Scan Services by HPLaserJetPrinter
Print Services by HPLaserJetPrinter
Fax Services by HPLaserJetPrinter
Print Duplex Services by HPLaserJetPrinter
Scan Services by LiquidInkjetPrinter
Print Services by LiquidInkjetPrinter

```

In the next article, I am going to discuss [Polymorphism in C#](#). Here, in this article, I try to explain **Multiple Inheritance RealTime Examples in C#**. I hope you enjoy this Multiple Inheritance in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this Multiple Inheritance Real-Time Examples in the C# article.